

Question:

Write a JAVA program to demonstrate the commonly used classes of the Java Collections

Framework: **ArrayList, LinkedList, Vector, Stack, HashSet, LinkedHashSet, TreeSet,**

PriorityQueue, ArrayDeque, HashMap, LinkedHashMap, TreeMap and Hashtable, using their commonly used methods.

Execution Steps:

Step 1: Program execution starts from main().

Step 2: **ArrayList**: "Java", "Python", "C", "Java" are added. add(1, "C++"), set(2, "HTML"), remove(1) and remove("C") change the list to [Java, HTML, Java]. size() = 3, isEmpty() = false, indexOf("Java") = 0 and lastIndexOf("Java") = 2. sort() gives [HTML, Java, Java].

Step 3: **LinkedList**: after add("B"), add("C"), addFirst("A") and addLast("D") the list is [A, B, C, D]. removeFirst() and removeLast() leave [B, C], offer("E") gives [B, C, E], peek() and poll() both return B and the list becomes [C, E].

Step 4: **Vector**: 10, 20 and 30 are added (add() and addElement()). set(1, 25) gives [10, 25, 30], remove(0) gives [25, 30] and removeElement(30) leaves [25]. size() = 1 and the default capacity() = 10.

Step 5: **Stack**: push(10), push(20) and push(30) give [10, 20, 30]. peek() = 30, search(20) = 2 (position from the top), pop() removes 30 and the stack becomes [10, 20].

Step 6: **HashSet**: the duplicate 20 is ignored. contains(20) is true, remove(10) leaves [20, 30], size() = 2.

Step 7: **LinkedHashSet**: insertion order is preserved and the duplicate "Java" is ignored: [Java, Python, C]. After remove("C") it is [Java, Python].

Step 8: **TreeSet**: elements are sorted: [10, 20, 30, 40]. first() = 10, last() = 40, higher(20) = 30, lower(20) = 10, ceiling(20) = 20 and floor(20) = 20. pollFirst() removes 10, pollLast() removes 40, leaving [20, 30].

Step 9: **PriorityQueue**: 30, 10 and 20 are added. peek() = 10 (smallest element is the head) and poll() removes it, leaving [20, 30].

Step 10: **ArrayDeque**: addFirst, addLast, offerFirst and offerLast give [10, 20, 30, 40]. peekFirst() = 10 and peekLast() = 40. pollFirst() and pollLast() remove both ends and leave [20, 30].

Step 11: **HashMap**: the keys 101, 102, 103 are mapped to Java, Python, C. get(101) = Java, containsKey and containsValue are true, getOrDefault(105, "Not Found") returns Not Found, and remove(103) leaves two entries.

Step 12: **LinkedHashMap**: insertion order is preserved: {1=One, 2=Two, 3=Three}. After remove(3) it is {1=One, 2=Two}.

Step 13: **TreeMap**: keys are sorted: {10=Ten, 20=Twenty, 30=Thirty, 40=Forty}. firstKey() = 10, lastKey() = 40, higherKey(20) = 30, lowerKey(20) = 10, ceilingKey(20) = 20 and floorKey(20) = 20.

Step 14: **Hashtable**: three entries are stored. get(2) = Banana, remove(3) leaves two entries and the Enumeration objects from keys() and elements() print the keys (2 1) and values (Banana Apple).

Step 15: main() ends and the program terminates.

Algorithm:

Step 1: Start.

Step 2: Import the java.util package.

Step 3: ArrayList: add, add(index), get, set, remove(index), remove(object), contains, size, isEmpty, indexOf, lastIndexOf and sort.

Step 4: LinkedList: add, addFirst, addLast, get, getFirst, getLast, removeFirst, removeLast, offer, peek and poll.

Step 5: Vector: add, addElement, get, set, remove, removeElement, size, capacity and contains.

Step 6: Stack: push, peek, search, pop and empty.

Step 7: HashSet: add (duplicates ignored), contains, remove, size and isEmpty.

Step 8: LinkedHashSet: add, contains, remove and size (insertion order is kept).

Step 9: TreeSet: add, first, last, higher, lower, ceiling, floor, pollFirst and pollLast (sorted order).

Step 10: PriorityQueue: add, offer, peek, poll, contains and size.

Step 11: ArrayDeque: addFirst, addLast, offerFirst, offerLast, peekFirst, peekLast, pollFirst and pollLast.

Step 12: HashMap: put, get, containsKey, containsValue, keySet, values, entrySet, size, getOrDefault and remove.

Step 13: LinkedHashMap: put, get, containsKey, keySet, values, entrySet and remove.

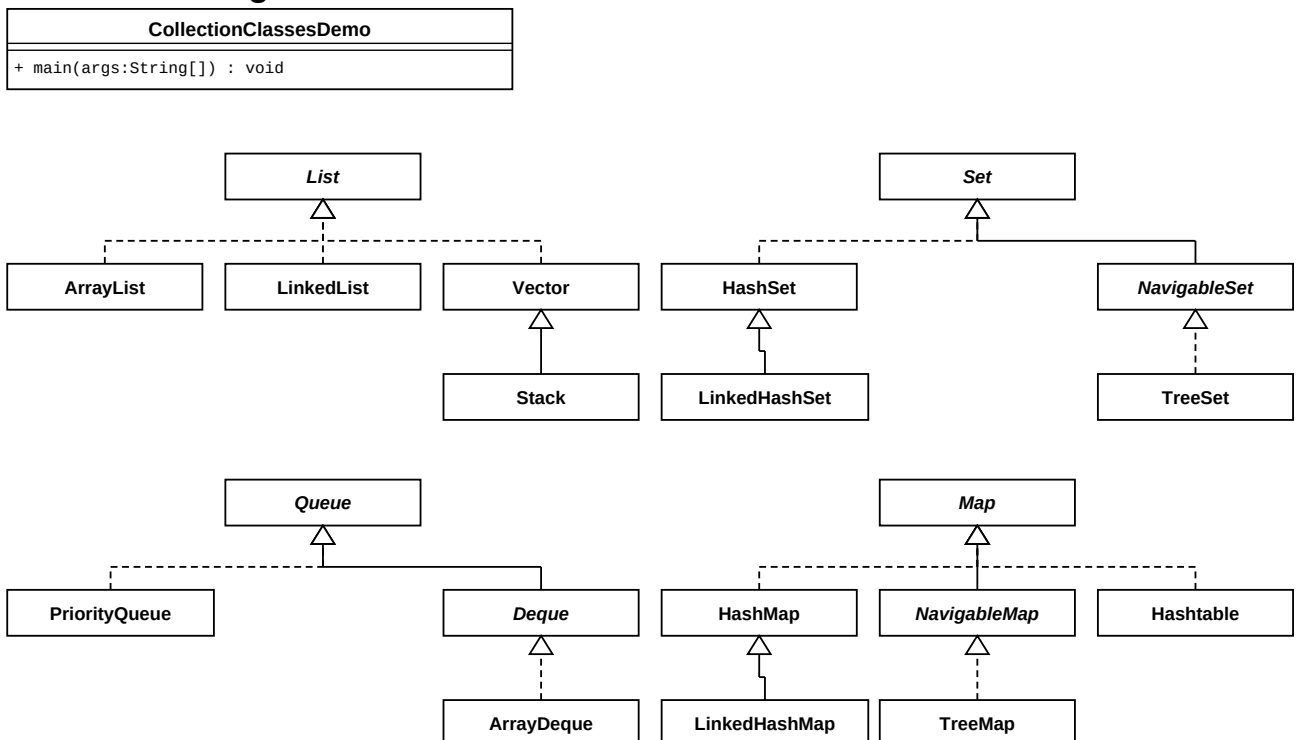
Step 14: TreeMap: put, get, firstKey, lastKey, higherKey, lowerKey, ceilingKey, floorKey, containsKey, containsValue and entrySet.

Step 15: Hashtable: put, get, containsKey, containsValue, size, remove, keys(), elements() and isEmpty.

Step 16: Display the result after each operation.

Step 17: Stop.

UML Class Diagram:



A solid line with a hollow triangle means 'extends' and a dashed line with a hollow triangle means 'implements'. Italic names are interfaces. All classes belong to `java.util` and are used by `main()` of `CollectionClassesDemo`.

Code:

```
import java.util.*;

public class CollectionClassesDemo {

    public static void main(String[] args) {

        // ===== ARRAYLIST =====
        System.out.println("===== ARRAYLIST =====");

        ArrayList<String> list = new ArrayList<>();

        list.add("Java");
        list.add("Python");
        list.add("C");
        list.add("Java");

        System.out.println("List: " + list);
        list.add(1, "C++");
        System.out.println("After add(index): " + list);
        System.out.println("Get index 2: " + list.get(2));

        list.set(2, "HTML");
        System.out.println("After set: " + list);

        list.remove(1);
        System.out.println("After remove(index): " + list);

        list.remove("C");
        System.out.println("After remove(object): " + list);

        System.out.println("Contains Java: " + list.contains("Java"));
        System.out.println("Size: " + list.size());
        System.out.println("Is Empty: " + list.isEmpty());
        System.out.println("First Index of Java: " + list.indexOf("Java"));
        System.out.println("Last Index of Java: " + list.lastIndexOf("Java"));

        list.sort(Comparator.naturalOrder());
        System.out.println("Sorted List: " + list);

        // ===== LINKEDLIST =====
        System.out.println("\n===== LINKEDLIST =====");

        LinkedList<String> linkedList = new LinkedList<>();

        linkedList.add("B");
        linkedList.add("C");
        linkedList.addFirst("A");
        linkedList.addLast("D");

        System.out.println("LinkedList: " + linkedList);
        System.out.println("Get index 1: " + linkedList.get(1));
        System.out.println("First: " + linkedList.getFirst());
        System.out.println("Last: " + linkedList.getLast());

        linkedList.removeFirst();
        linkedList.removeLast();

        System.out.println("After removeFirst/removeLast: " + linkedList);

        linkedList.offer("E");
        System.out.println("After offer: " + linkedList);

        System.out.println("Peek: " + linkedList.peek());
        System.out.println("Poll: " + linkedList.poll());
        System.out.println("After poll: " + linkedList);
        // ===== VECTOR =====
        System.out.println("\n===== VECTOR =====");
```

```

Vector<Integer> vector = new Vector<>();

vector.add(10);
vector.add(20);
vector.addElement(30);

System.out.println("Vector: " + vector);
System.out.println("Get index 1: " + vector.get(1));

vector.set(1, 25);
System.out.println("After set: " + vector);

vector.remove(0);
System.out.println("After remove: " + vector);

vector.removeElement(30);
System.out.println("After removeElement: " + vector);

System.out.println("Size: " + vector.size());
System.out.println("Capacity: " + vector.capacity());
System.out.println("Contains 25: " + vector.contains(25));

// ===== STACK =====
System.out.println("\n===== STACK =====");

Stack<Integer> stack = new Stack<>();

stack.push(10);
stack.push(20);
stack.push(30);

System.out.println("Stack: " + stack);
System.out.println("Peek: " + stack.peek());
System.out.println("Search 20: " + stack.search(20));

System.out.println("Pop: " + stack.pop());
System.out.println("After pop: " + stack);
System.out.println("Is Empty: " + stack.empty());

// ===== HASHSET =====
System.out.println("\n===== HASHSET =====");

HashSet<Integer> hashSet = new HashSet<>();

hashSet.add(10);
hashSet.add(20);
hashSet.add(30);
hashSet.add(20);

System.out.println("HashSet: " + hashSet);
System.out.println("Contains 20: " + hashSet.contains(20));

hashSet.remove(10);
System.out.println("After remove: " + hashSet);

System.out.println("Size: " + hashSet.size());
System.out.println("Is Empty: " + hashSet.isEmpty());

// ===== LINKEDHASHSET =====
System.out.println("\n===== LINKEDHASHSET =====");

LinkedHashSet<String> linkedHashSet = new LinkedHashSet<>();

linkedHashSet.add("Java");
linkedHashSet.add("Python");
linkedHashSet.add("C");

```

```

linkedHashSet.add("Java");

System.out.println("LinkedHashSet: " + linkedHashSet);
System.out.println("Contains Python: "
    + linkedHashSet.contains("Python"));

linkedHashSet.remove("C");
System.out.println("After remove: " + linkedHashSet);
System.out.println("Size: " + linkedHashSet.size());

// ===== TREESSET =====
System.out.println("\n===== TREESSET =====");

TreeSet<Integer> treeSet = new TreeSet<>();

treeSet.add(30);
treeSet.add(10);
treeSet.add(40);
treeSet.add(20);

System.out.println("TreeSet: " + treeSet);
System.out.println("First: " + treeSet.first());
System.out.println("Last: " + treeSet.last());
System.out.println("Higher than 20: " + treeSet.higher(20));
System.out.println("Lower than 20: " + treeSet.lower(20));
System.out.println("Ceiling of 20: " + treeSet.ceiling(20));
System.out.println("Floor of 20: " + treeSet.floor(20));

System.out.println("Poll First: " + treeSet.pollFirst());
System.out.println("Poll Last: " + treeSet.pollLast());
System.out.println("After polling: " + treeSet);

// ===== PRIORITYQUEUE =====
System.out.println("\n===== PRIORITYQUEUE =====");

PriorityQueue<Integer> priorityQueue = new PriorityQueue<>();

priorityQueue.add(30);
priorityQueue.offer(10);
priorityQueue.offer(20);

System.out.println("PriorityQueue: " + priorityQueue);
System.out.println("Peek: " + priorityQueue.peek());
System.out.println("Poll: " + priorityQueue.poll());
System.out.println("After poll: " + priorityQueue);
System.out.println("Contains 20: " + priorityQueue.contains(20));
System.out.println("Size: " + priorityQueue.size());

// ===== ARRAYDEQUE =====
System.out.println("\n===== ARRAYDEQUE =====");

ArrayDeque<Integer> deque = new ArrayDeque<>();

deque.addFirst(20);
deque.addLast(30);
deque.offerFirst(10);
deque.offerLast(40);

System.out.println("ArrayDeque: " + deque);
System.out.println("Peek First: " + deque.peekFirst());
System.out.println("Peek Last: " + deque.peekLast());

System.out.println("Poll First: " + deque.pollFirst());
System.out.println("Poll Last: " + deque.pollLast());

System.out.println("After polling: " + deque);

```

```
// ===== HASHMAP =====
System.out.println("\n===== HASHMAP =====");

HashMap<Integer, String> hashMap = new HashMap<>();

hashMap.put(101, "Java");
hashMap.put(102, "Python");
hashMap.put(103, "C");

System.out.println("HashMap: " + hashMap);
System.out.println("Get key 101: " + hashMap.get(101));
System.out.println("Contains key 102: "
    + hashMap.containsKey(102));
System.out.println("Contains value Python: "
    + hashMap.containsValue("Python"));
System.out.println("Keys: " + hashMap.keySet());
System.out.println("Values: " + hashMap.values());
System.out.println("Entries: " + hashMap.entrySet());
System.out.println("Size: " + hashMap.size());

System.out.println("Get missing key: "
    + hashMap.getOrDefault(105, "Not Found"));

hashMap.remove(103);
System.out.println("After remove: " + hashMap);

// ===== LINKEDHASHMAP =====
System.out.println("\n===== LINKEDHASHMAP =====");

LinkedHashMap<Integer, String> linkedHashMap =
    new LinkedHashMap<>();

linkedHashMap.put(1, "One");
linkedHashMap.put(2, "Two");
linkedHashMap.put(3, "Three");

System.out.println("LinkedHashMap: " + linkedHashMap);
System.out.println("Get key 2: " + linkedHashMap.get(2));
System.out.println("Contains key 1: "
    + linkedHashMap.containsKey(1));
System.out.println("Keys: " + linkedHashMap.keySet());
System.out.println("Values: " + linkedHashMap.values());
System.out.println("Entries: " + linkedHashMap.entrySet());

linkedHashMap.remove(3);
System.out.println("After remove: " + linkedHashMap);

// ===== TREEMAP =====
System.out.println("\n===== TREEMAP =====");

TreeMap<Integer, String> treeMap = new TreeMap<>();

treeMap.put(30, "Thirty");
treeMap.put(10, "Ten");
treeMap.put(40, "Forty");
treeMap.put(20, "Twenty");

System.out.println("TreeMap: " + treeMap);
System.out.println("Get key 20: " + treeMap.get(20));
System.out.println("First Key: " + treeMap.firstKey());
System.out.println("Last Key: " + treeMap.lastKey());

System.out.println("Higher Key than 20: "
    + treeMap.higherKey(20));
System.out.println("Lower Key than 20: "
    + treeMap.lowerKey(20));
System.out.println("Ceiling Key of 20: "
    + treeMap.ceilingKey(20));
```

```

        + treeMap.ceilingKey(20));
System.out.println("Floor Key of 20: "
        + treeMap.floorKey(20));

System.out.println("Contains key 30: "
        + treeMap.containsKey(30));
System.out.println("Contains value Ten: "
        + treeMap.containsValue("Ten"));
System.out.println("Entries: " + treeMap.entrySet());

// ===== HASHTABLE =====
System.out.println("\n===== HASHTABLE =====");

Hashtable<Integer, String> hashtable = new Hashtable<>();

hashtable.put(1, "Apple");
hashtable.put(2, "Banana");
hashtable.put(3, "Mango");

System.out.println("Hashtable: " + hashtable);
System.out.println("Get key 2: " + hashtable.get(2));

System.out.println("Contains key 1: "
        + hashtable.containsKey(1));

System.out.println("Contains value Mango: "
        + hashtable.containsValue("Mango"));

System.out.println("Size: " + hashtable.size());

hashtable.remove(3);
System.out.println("After remove: " + hashtable);

System.out.println("Keys: ");

Enumeration<Integer> keys = hashtable.keys();

while (keys.hasMoreElements()) {
    System.out.print(keys.nextElement() + " ");
}

System.out.println("\nValues: ");

Enumeration<String> values = hashtable.elements();

while (values.hasMoreElements()) {
    System.out.print(values.nextElement() + " ");
}

System.out.println();
System.out.println("Is Empty: " + hashtable.isEmpty());
    }
}

```

Output:

```
===== ARRAYLIST =====
List: [Java, Python, C, Java]
After add(index): [Java, C++, Python, C, Java]
Get index 2: Python
After set: [Java, C++, HTML, C, Java]
After remove(index): [Java, HTML, C, Java]
After remove(object): [Java, HTML, Java]
Contains Java: true
Size: 3
Is Empty: false
First Index of Java: 0
Last Index of Java: 2
Sorted List: [HTML, Java, Java]

===== LINKEDLIST =====
LinkedList: [A, B, C, D]
Get index 1: B
First: A
Last: D
After removeFirst/removeLast: [B, C]
After offer: [B, C, E]
Peek: B
Poll: B
After poll: [C, E]

===== VECTOR =====
Vector: [10, 20, 30]
Get index 1: 20
After set: [10, 25, 30]
After remove: [25, 30]
After removeElement: [25]
Size: 1
Capacity: 10
Contains 25: true

===== STACK =====
Stack: [10, 20, 30]
Peek: 30
Search 20: 2
Pop: 30
After pop: [10, 20]
Is Empty: false

===== HASHSET =====
HashSet: [20, 10, 30]
Contains 20: true
After remove: [20, 30]
Size: 2
Is Empty: false

===== LINKEDHASHSET =====
LinkedHashSet: [Java, Python, C]
Contains Python: true
After remove: [Java, Python]
Size: 2

===== TREESSET =====
TreeSet: [10, 20, 30, 40]
First: 10
Last: 40
Higher than 20: 30
Lower than 20: 10
Ceiling of 20: 20
```

```
Floor of 20: 20
Poll First: 10
Poll Last: 40
After polling: [20, 30]

===== PRIORITYQUEUE =====
PriorityQueue: [10, 30, 20]
Peek: 10
Poll: 10
After poll: [20, 30]
Contains 20: true
Size: 2

===== ARRAYDEQUE =====
ArrayDeque: [10, 20, 30, 40]
Peek First: 10
Peek Last: 40
Poll First: 10
Poll Last: 40
After polling: [20, 30]

===== HASHMAP =====
HashMap: {101=Java, 102=Python, 103=C}
Get key 101: Java
Contains key 102: true
Contains value Python: true
Keys: [101, 102, 103]
Values: [Java, Python, C]
Entries: [101=Java, 102=Python, 103=C]
Size: 3
Get missing key: Not Found
After remove: {101=Java, 102=Python}

===== LINKEDHASHMAP =====
LinkedHashMap: {1=One, 2=Two, 3=Three}
Get key 2: Two
Contains key 1: true
Keys: [1, 2, 3]
Values: [One, Two, Three]
Entries: [1=One, 2=Two, 3=Three]
After remove: {1=One, 2=Two}

===== TREEMAP =====
TreeMap: {10=Ten, 20=Twenty, 30=Thirty, 40=Forty}
Get key 20: Twenty
First Key: 10
Last Key: 40
Higher Key than 20: 30
Lower Key than 20: 10
Ceiling Key of 20: 20
Floor Key of 20: 20
Contains key 30: true
Contains value Ten: true
Entries: [10=Ten, 20=Twenty, 30=Thirty, 40=Forty]

===== HASHTABLE =====
Hashtable: {3=Mango, 2=Banana, 1=Apple}
Get key 2: Banana
Contains key 1: true
Contains value Mango: true
Size: 3
After remove: {2=Banana, 1=Apple}
Keys:
2 1
Values:
Banana Apple
```

Is Empty: false